

硅 碳 协 同 · 效 能 倍 增

 百融说 **AI Native人才分享**



CLAUDE CODE · SHARING SESSION

一人多栈

Claude Code 驱动 Java / Go/ Rust / Python 全栈协同开发

autobots工作台协同实战 · 2026.04

今天讲什么

- | | | |
|----|----|---|
| 01 | 盘子 | 一人 4 栈的质量负荷
4 栈 × 4 环境 × 15 模块 · 一个人凭什么保质量 |
| 02 | 痛点 | 多栈的真实质量代价
注意力切碎 / 契约漂移 / 坑踩两遍 / 长任务断档 |
| 03 | 方法 | 三件套 = 三把质量尺
Memory 锁纪律 · Sub-agent 并行审 · Skill+Hook 闸门续跑 |
| 04 | 存 | 纪律怎么存：4 层金字塔
项目 CLAUDE.md · 全局 rules · memory · skill+hook |
| 05 | 跑 | 纪律怎么跑：跨栈并行 + 续跑闭环
5 条泳道 · feature 链路 · preset · ralph-loop + Hook |
| 06 | 验 | 纪律怎么验：度量 · 契约 · 反模式
数字对账 · Java/Go/Python/TS 契约 · 避坑清单 |
| 07 | 收 | 核心观点 & Q&A
质量 = 纪律 × AI 执行 × Hook 兜底 |

一个人的盘子

4

PROD LANGUAGE
STACKS

Java · Go · Python ·

TS/React · Rust

4

ENVIRONMENTS

dev · pre · poc · prod 各
有独立配置

15

PLATFORM
MODULES

auth · agent ·

store · observe ·

tenant · discovery ...

6+

INFRA
COMPONENTS

Nacos · MySQL ·

Redis · MinIO ·

LanceDB · Grafana

Frontend React 18 + TS → **Gateway** Spring Cloud → **Core** Java 21 × 4 微服务

Orchestrator Go 1.21 → **Runtime** Python (legacy) · Agent 协议层 → **Infra** Nacos 统配

一人多栈，不是"烦"，是容易错

切语言比切分支贵得多 — 真正的代价不是时间，是质量

1 注意力被切碎

QUALITY COST · 盲区

Java 跳 Go 再跳 TS — 脑子每切一次语言，**之前栈的检查项容易漏一个**。单人审查容易留盲区

2 跨栈契约会漂

QUALITY COST · 漂移

Java 改字段 → Python 还在用旧版 → TS 类型报错 → Go 透传错误。**四栈对齐靠人眼 = 靠不住**

3 同一个坑会踩两遍

QUALITY COST · 重复事故

上次 Flyway 在 pre 翻过车。今天再写 migration 还是同样的配置 — **规则没落地的纪律等于没纪律**

4 长任务中途断档

QUALITY COST · 半截工程

跨 4 栈统一 error code 这种活，一次写不完。**session 结束 = 任务断档 = 对齐不全**

三件套 = 三把质量尺

纪律要被存下来、被执行、被验收 — 三件套各管一件

01 / MEMORY

锁住纪律

纪律的"存储层"

踩过的坑、对过的齐、定过的规则
— 一次写入，所有后续会话受益

SOLVES

同一个坑不再踩第二遍 · 跨栈状态 AI 不乱动

02 / SUB-AGENT

并行审查

纪律的"执行层"

多专家并行审 — security / build / db
/ e2e 各盯一块 · 人只做调度和裁决

SOLVES

单人注意力盲区 · 跨栈契约漂移 · 分支并跑冲突

03 / SKILL + HOOK

闸门 + 续跑

纪律的"验收层"

ralph-loop 用 promise 定"什么算完" ·
Hook 在 toolCall 时自动拦截格式/安全/调试痕迹

SOLVES

长任务中途断档 · console.log 混入 ·
push 未审先发

纪律怎么存：四层约束金字塔

AI 不是自觉讲质量 — 是你把质量规则“存”到它能看见的地方

1

PROJECT CLAUDE.md · 仓库级 · 随 git 版本

项目根目录 **.JCLAUDE.md** — 4 栈架构 / 端口 / Nacos 命名空间 / 部署规则。团队人人可见，人人可改

2

GLOBAL RULES · 用户级 · 私人纪律

全局 **~/.claude/rules/*.md** — coding-style / git-workflow / security / testing 跨项目通用纪律

3

MEMORY · 会话跨轮 · 自动增长

~/.claude/projects/.../memory/ 下 12 feedback + 19 project + 3 其他 = **34 条索引** — 每踩一次坑就多一条约束

4

SKILL + HOOKS · 按需 + 自动

Skill 描述匹配时注入 (ralph-loop/qa/ship) · Hook 在特定时刻自动拦截 (PreToolUse / Stop)

为什么这是金字塔：1-2 层变化慢 · 3 层随项目增长 · 4 层每次动作即时执行。越底层越慢越稳，越顶层越快越精准。

CLAUDE.md 三层原文：每次对话自动注入

规则靠手写 — 每层原文都是从实际文件抠出来的

L1 · GLOBAL

~/ .claude/CLAUDE.md
~350 行 · 所有项目共用

```
# 智能体生态系统 - 精简调度器 v3.0
## 🧠 核心原则 (必须遵守)
1. **第一性原理**: 回归问题本质推导方案
2. **DRY**: 避免重复代码 · 3. **KISS**: 保持简单
5. **YAGNI**: 不实现当前不需要的功能
6. **500行规则**: 超过500行必须拆分
7. **Python项目**: 统一使用 `uv`
8. **错误暴露**: 积极暴露错误 · 拒绝回退
```

L2 · PROJECT

agplateform/CLAUDE.md
~10KB · 本仓库专属

```
## Project Overview
Autobots Platform is an enterprise Agent collaboration
platform built on AgentScope SDK. Multi-language
architecture where each layer uses the most
appropriate technology.
Configuration loading priority:
`Nacos > application-{env}.yaml > application.yaml`
### Git Workflow
- Branch from `dev`, merge back to `dev`
- Commit format: `(<scope>): <subject>`
```

L3 · SUBDIR

frontend/CLAUDE.md
扩展点

```
#本项目暂未启用 subdir CLAUDE.md
#下面是未来可能的形态 ( 仅为示例 ) :
// frontend/CLAUDE.md
- 组件命名: PascalCase · 文件名 kebab-case
- 所有 API 调用走 src/api/ 封装, 禁 axios 直调
// rust/CLAUDE.md
- `cargo clippy -- -D warnings` 必须过
- `unsafe` 块必须有 SAFETY 注释 + reviewer 二人
```

CLAUDE.MD (规则)

手写 · 上来就该知道 · 不变/慢变 · 版本库管理

MEMORY (经验)

沉淀 · 踩过才知道 · 随事故新增 · memory/独立目录

HOOK (兜底)

执行期 · AI 忘了也挡得住 · settings.json 里的 shell 命令

沉淀团队 SOP 的第一性原理

让 AI 记住你团队的规矩，比每次重说一遍划算得多

为什么需要 Memory

Agent 默认是"金鱼"，每次对话都是新生。

你给的纪律（"部署怎么走"、"合并用 merge"）一次就忘。第 N 次解释 = 成本 × N。

Memory 的本质

一组 markdown 文件 + 索引，每次对话自动加载 MEMORY.md。

Agent 按需读具体文件。不是"超长 prompt"，是"按主题分文件的外部记忆"。

四种记忆类型

USER

用户的角色、目标、专长 — 让 AI 用你的语言讲话

FEEDBACK

纠正 & 确认 — 团队纪律的核心载体，带 Why + How to apply

PROJECT

项目专属事实 — 环境连接串、业务约束、在进行的迁移

REFERENCE

外部系统指针 — Jira 项目、Grafana 面板、Nacos 命名空间

Memory：锁住跨栈共享的"上下文"

4 栈要协同 · 先让 AI 知道每个栈在哪个状态

THE PROBLEM

Java Core 活跃维护 · Go Orchestrator 限定在编排域 · Python runtime 维护态 · TS 跟 API — AI 默认不知道每栈的纪律和环境

栈	当前状态与纪律	MEMORY 条目
Java	主干活跃：权限 / 租户 / 存储层持续迭代	feedback_autobots_is_ours.md — 平台主干
Go	只动 orchestrator，不跨域；workflow 为主	project_error_codes_unfinished_20260409.md
Python	runtime 维护态 · 只修 bug 不加功能	project_simplemem_bugs.md — bug fix only
TS / React	3 面板配置 + Admin + ChatPanel 基准	frontend_agent_config_ui.md · frontend_patterns.md
跨栈共享	4 环境的 Nacos + MySQL + Redis 连接信息	project_pre_nacos / pre_database / pre_redis_proxy

Memory 的多栈价值：不是记笔记 · 是让 AI 改任意一栈时都知道 —— 其他 3 栈此刻是活跃 / 维护 / 冻结？能不能联动？哪些接口不能动？

"部署" 一句话，走完整链路

一个 *feedback memory*，省下未来几十次的重复解释

USER

> 部署一下 auth-service

FEEDBACK_DEPLOY_WORKFLOW.MD

规则 · 4 步 SOP :

- ① commit & push dev →
- ② merge dev→main push
- ③ 检查 Nacos/MySQL 同步 pre →
- ④ 列出需重启 pre 容器

Why: 历史上漏过 Nacos 同步，pre 环境接口 500

边界: K8s 重启 agent 不碰，由人执行

Agent 展开时联动的其他纪律

01 git status / diff 确认改动清单

02 merge 不 reset (feedback_merge_not_push.md 约束)

03 先推 tencent/dev, origin 等用户说"部署"
(feedback_push_order.md)

04 MySQL schema 直读 information_schema
(feedback_no_flyway.md)

05 列容器清单，K8s 重启人工执行
(feedback_deploy_scope.md)

每一条 memory，都是一次事故的账单

feedback memory 不是纸面规则，是“踩过才写进去”的血泪账

1

feedback_no_flyway FEEDBACK · 2026

WHERE IT HURT

Flyway 迁移在 pre/poc/prod 不同步 · AI 看 migration 文件推断错了 schema

THE RULE

全环境禁用 Flyway · 直接对比 information_schema

WHAT IT SAVES

再也不用对 AI 解释“看 migration 文件是陷阱”

2

project_pre_redis_proxy PROJECT · 2026

WHERE IT HURT

Pre 环境 Redis 是代理型 · AI 执行 KEYS/SCAN 触发告警

THE RULE

禁 KEYS/SCAN · 扫描类命令全部绕开

WHAT IT SAVES

一次告警一次事故，AI 再碰 pre 环境直接避

3

feedback_prod_ip_allowlist FEEDBACK · 2026

WHERE IT HURT

生产 allowlist 含 16.0.0.0/8 · 上次差点被当成笔误删掉

THE RULE

这是 K8s Pod CIDR · 别删

WHAT IT SAVES

AI 再看到这段 IP 知道“是对的”，不会推荐删除

4

feedback_deploy_workflow FEEDBACK · 2026

WHERE IT HURT

“部署”漏做 Nacos 同步 · pre 环境接口 500

THE RULE

“部署” = commit → merge → nacos → mysql → 列容器

WHAT IT SAVES

一句“部署”能跑完整链路 · 不再漏步

加一条 memory : feedback_no_flyway.md 真实原文

不是"让 AI 记笔记", 是"把这次踩坑结晶成下次的自动纪律"

REAL · feedback_no_flyway.md VERBATIM

```
// ① 文件名: feedback_<主题>.md
// ② frontmatter (name / description / type)
---
name: 用户已禁用Flyway 不要再提
description: pre/poc/prod 都禁用了 Flyway,
schema 同步必须直接对比表结构·不要看 migration 文件
type: feedback
originSessionId: dl299ae8-974a-4f88...
---
用户已经在 pre / poc / prod 全部禁用 Flyway
(Spring.flyway.enabled=false), 不要再提 Flyway
自动迁移·不要看 db/migration/*.sql 文件。
// ③ Why - 决策根因 (让未来判断是否仍适用)
**Why:** 用户有自己的发版/数据库变更流程·
多次纠正过这一点 (2026-04-13 同一次会话已经被
纠正两次)。继续提 Flyway = 没听用户的话。
// ④ How to apply - 触发条件 (让未来知道何时拿出来用)
**How to apply:**
- 同步 schema 时直接用 `information_schema.columns`
/ `statistics` 对比两个库的表/列/索引差异
- 发现差异后用 `SHOW CREATE TABLE` 拉 DDL 手动 apply
- 不要说 "Flyway 会自动跑"、"你需要 V96 migration"
- 不要在响应里出现 "Flyway" 这个词·除非用户先提
```

STEP 01 · FILE

feedback_no_flyway.md

STEP 02 · CONTENT

frontmatter + Why + How to apply

STEP 03 · INDEX

追加 MEMORY.md 一行

REAL · MEMORY.md 第 21 行 VERBATIM

- [feedback_no_flyway.md] - Flyway 已全环境禁用
， schema 同步直接对比 information_schema, 不看
migration 文件

TIMELINE · 一条 memory 的一生

04-13 事故· AI 读 db/migration/*.sql 推断 schema
04-13 复发· 同会话 AI 换说法又提 Flyway
04-13 落盘· 写文件· 追加索引
04-13+ 回报· 新会话零解释成本

触发写 memory 的信号: 同会话被纠正 ≥ 2 次·或每次都要解释同一件事·或事故后的隐性约束。规则: 纠正一次是对话, 纠正两次是 memory。

其他 memory 样本：4 种类型 × 4 种形态

memory 不是只有“踩坑纠正”一种 — 还有环境约束、品牌决策、技术栈偏好

TYPE · FEEDBACK · 纠正

feedback_no_frontend_defaults.md

前端页面不该有任何默认值回填逻辑，
加载到什么就显示什么。

Why: 默认值 fallback 会覆盖用户的主动

清空操作（如解绑知识库）→ 功能 bug。

How: 空数组 / 空字符串是用户的合法
输入，表单初始化/序列化不回填。

TYPE · PROJECT · 环境约束

project_pre_redis_proxy.md

Pre 环境 Redis 是代理型，禁 KEYS / SCAN。
任何调用 → JedisDataException → 启动失败。

Why: 04-13 SliderCaptcha bootstrap 用
redisTemplate.keys("slider:*") pre 直接挂。
dev 是单机 Redis 能跑 → 本地测不发现。

How: keys/scan 调用都是 pre 地雷，
review 遇到 redisTemplate.keys 直接 reject

TYPE · PROJECT · 策略

project_branding.md

平台正式名称定为“autobots工作台”。

Why: 用户决定的品牌命名，取代

Agentic Platform / AP平台 等称呼。

How: 讨论 / 文档 / UI 文案用“鲁班工作台”；
代码模块名 agentic-platform 保持不变，
除非用户明确要求修改。

TYPE · USER · 技术栈偏好

user_frontend_arch.md

Framework: React 18 + TS + pnpm + Vite + AntD

Agent Edit: 3-panel (main | tabs | chat)

Tokens: Primary #165DFF · text #1D2129 ·

border #E5E6EB · page bg #F7F8FA

MainEdit: nameRow + promptSection (flex:1)
+ modelBar (model/temp/topP/maxTokens)

Font: Inter + DM Mono

跨类型共同点：每条都有 **Why**（决策根因，判断仍适用）和 **How**（触发条件，判断何时启用）。**长度差 30×**（branding 3 行 vs rust_runtime_waves 400+ 行），**一视同仁地加载。**

跨栈并行：一次 **commit**，五种审查同时跑

改完 auth 后自动派活 — 不是我一条条等，是 5 条泳道一起游

场景：改了 platform-auth 的 token 签发 → 同时影响 Java Core / Go Orchestrator / Python 兼容层 / TS 前端 / Nacos 配置

JAVA lane

security-reviewer · OWASP + Spring Boot 鉴权链 + sa-token 会话一致性

GO lane

build-error-resolver · go build + go test · workflow 调用链不断

PYTHON lane

Explore + pytest · runtime 兼容性检查 · 新 token 格式不破坏旧会话

TS lane

e2e-runner · Playwright 跑登录/权限/ChatPanel 流程

DB + DOC lane

database-reviewer + doc-updater · schema 对齐 · Nacos yaml + codemap

TOP 5 REAL AGENT CALLS · 本项目 772 次累计

312

Explore
跨栈侦查

203

general
综合任务

98

team-impl
并行实施

35

Plan
跨栈规划

27

team-debug
跨栈排障

不用想象，看数据

本项目 214 个 session · 过去几个月真实调用分布

791

TOTAL AGENT INVOCATIONS

Claude Code 在本项目累计发起的 sub-agent 调用次数

Explore

310 · 40%

general-purpose

194 · 25%

team-implementer

98 · 12%

Plan

35 · 4%

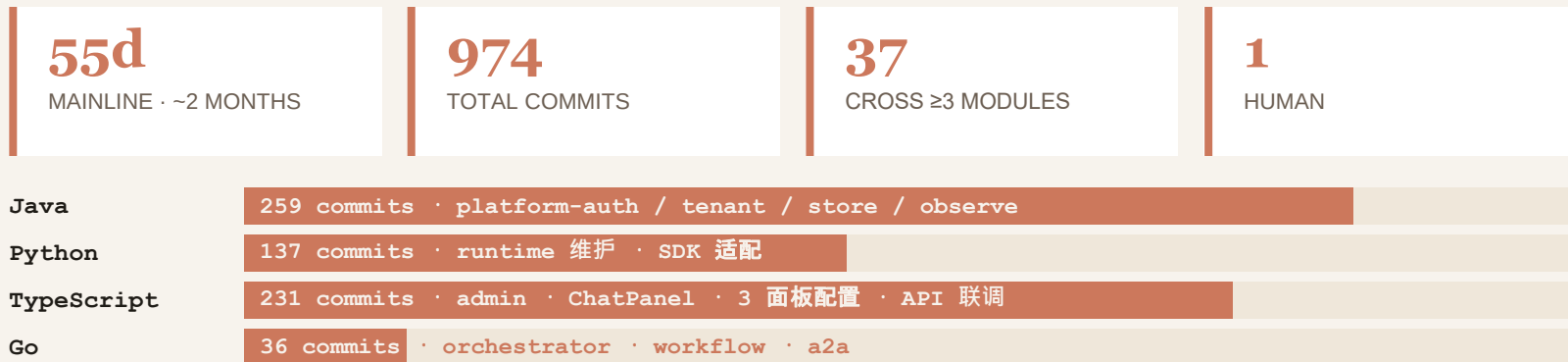
team-debugger

27 · 3%

注意那两条深色的 — team-implementer 98 次 + team-debugger 27 次 = 125 次团队调用 · 占 15%。

4 栈的质量负荷 · 主干全周期数据

2026-02-26 至今, 2 个月不到, 凭什么一个人做完还能保质量?



典型跨栈 COMMIT · 真实数据

6334e217 知识库: Java 核心 + Python 运行 + TS 面板 · 3 栈联动

9094e084 admin 面板: Java schema + TS × 3 · 2 栈

7b08c36b 图像 KB: 纯 Python 15 个文件 + JSON 配置 · 单栈大改

2026-04-19 · 我的一天

这不是假设，是 `git log` 里真实的一天 — 一个人，五种语言

191

COMMITs · IN 24H

5

LANGUAGES TOUCHED

~20

PARALLEL AGENT TURNS

1

HUMAN

Rust

`ap-runtime / ap-store / ap-bailian` · Wave 批次

Java

`platform-auth / tenant / store` 权限与租户隔离

Python

`legacy runtime` · parity 对标

TypeScript

`admin 面板` · ChatPanel 基准

Go

`orchestrator` · workflow

典型跨栈 COMMIT 链

`b23dc25c` `auth + tenant + store` → 3 platform 模块 (Java)

`0d250cf0` `Java tenant handlers + Rust tenant` → 租户隔离跨栈

`9094e084` `admin 面板 TS × 3 + Java × 1` → 全栈小闭环

一次"知识库引用"的跨栈链路

真实 commit 6334e217 · 一次 feature 牵动 Java + Python + TS 三栈



REAL COMMIT TRAIL · 近一周

6334e217 chat 引用可点 · Java + Python + TS [3 栈联动]

9535dca3 Feign BaigongClient 负载均衡修复 · Java 单栈

7b08c36b 图搜图 · 纯 Python 15 file [单栈大改]

没 AI 的话：这条链至少 1.5 天。有 AI：每步的 reviewer / pytest / e2e 都并行，一人多栈的真实解法不是同时写 4 种语言，是同时发 4 条审查。

4 个 preset, 4 种质量保障场景

不是新概念 — 是给已经在做的质量活儿配上专业团队

PRESET	守住的质量 · 我的真实场景	怎么并行
review 5 维度并行 reviewer	跨栈契约一致性 · drift 早发现 场景：Java API ↔ Python 兼容层 ↔ TS 类型	security / types / behavior / edge / docs 五 维同时扫
fullstack Java + Python + TS + Go	跨栈 feature 质量 · 集成冲突前置 场景：一次 feature 跨 ≥3 模块 · 37 次 / 55 天主干	文件所有权锁死 ：每个 worker 只能动自己 分得的文件
debug 竞争假设 debugger ×N	跨栈故障定位 · 降低漏判 场景：前端报错 → 根因在 Java / Python / Go?	3 条假设 并行验证 · 不会走偏到死胡同
security OWASP + auth + deps + secrets	认证链路审计 · 跨栈鉴权一致 场景：Gateway + auth-service + sa-token + 前端	4 维度并行扫描 · 覆盖 Nacos → token 存储 主路径

质量门槛：不是技术门槛，是纪律门槛 — 先画清 4 栈文件所有权，agent 团队才敢并跑。

Skill：按需加载的能力插件

Memory 记状态 · Sub-agent 委派任务 · Skill 补第三条腿 — 能力

MEMORY

记住状态

跨会话持久。团队纪律、环境连接串、决策历史

SUB-AGENT

委派任务

独立上下文的专家。主 agent 只调度，子 agent 只深挖

SKILL

装载能力

一次性加载的"说明书 + 脚本包"。用完即走，不占持久空间

PROGRESSIVE DISCLOSURE · 按描述触发三段加载

01 · SENSE

匹配 skill 的 description 触发条件



02 · LOAD

把 SKILL.md + 脚本路径注入主上下文



03 · EXECUTE

Agent 按流程走，调用 bundled 脚本

这套 PPT 就是 Skill 造的

最好的例子总是现场的那一个

THIS SESSION · REAL INVOCATION

Skill(**document-skills:pptx**) → 加载 html2pptx.md + ooxml.md + scripts/
→ 我按规范写 15 个 HTML slide → node build.js → **17 张 pptx 交付**

本次实际触发

document-skills:pptx — HTML→PPTX 工作流，带校验和 bundled 脚本

agent-teams (team-spawn / SendMessage) — 本次派 3 个 Explore 并行调研用的就是它

项目常备 · 可按需触发

coding-standards — TS/JS/React/Node 编码规范

security-review — OWASP 检查 + 鉴权/密钥流程

tdd-workflow — 红绿重构 + 80% 覆盖率闸门

backend-patterns — Express/Next.js API 设计

architecture-analyzer — 基于第一性原理的架构诊断

Skill 的价值：不是“记住要用什么工具”，而是“当任务匹配时，工具自己来找我”。Memory 记的是 WHO / WHAT，Skill 管的是 HOW。

ralph-loop : 质量闸门 + 续跑引擎

本项目 30 次启动 · 12 次 cancel · 60% 自收敛 — 真正被当主力的 skill 就这一个

EXECUTION LOOP

01 · START

定 promise

写 state 文件 · iteration / max
/ completion_promise



02 · RUN

AI 跑一轮

按 prompt 干活 · 结束时试图
退出



03 · HOOK

Stop hook 拦截

读 transcript · perl 非贪婪抽
<promise> 标签



04 · JUDGE

字面对比

相等 → 放行收工 · 不等 →
block + 注入下轮

THE KEY TRICK

```
$ /ralph-loop --max-iterations 20 --completion-promise "error code 跨 4 语言统一"
```

↓ 写 `.claude/ralph-loop.local.md` (YAML frontmatter + prompt 正文)

↓ AI 每轮结束输出 `<promise>...</promise>` 标签

↓ hook 用 `perl` 非贪婪匹配抽标签内容 = `completion_promise?`

↓ 不等: `jq` 输出 `{"decision": "block", "reason": <prompt>}` 注入下轮

为什么是质量闸门: `completion-promise` = 你定义的"什么才算做完"。AI 自己觉得完了没用, `promise` 字面对上才算完。人不在场、质量标准不妥协。

Hook：让纪律不靠 AI 自觉

AI 可能忘，但 hook 不会 — 这是质量的最后一道闸

BEFORE TOOL CALL

PreToolUse

tmux reminder
npm / mvn 等长命令建议走 tmux

git push review
push 前打开 Zed 让人再看一眼

doc blocker
拦截未授权的 .md / .txt 创建

AFTER TOOL CALL

PostToolUse

Prettier auto-format
TS/JS 编辑后自动格式化

tsc type check
.ts/.tsx 编辑后跑类型检查

console.log warn
检测到 console.log 立刻告警

WHEN SESSION ENDS

Stop

console.log audit
会话结束扫全部改动 是否遗漏 debug 语句

ralph-loop
promise 未满足则阻止退出 · 注入下轮

context save
session 摘要落盘 · 下次恢复

REAL EXAMPLE · HOOK 怎么写

```
~/.claude/settings.json → hooks → PostToolUse  
match: "Edit|Write" + filePath: "**/*.{ts,tsx}"  
command: "prettier --write $CLAUDE_FILE_PATHS"
```

关键：memory 是“告诉 AI 什么是对的”，hook 是“即使 AI 忘了，也拦住”。两层兜底，质量才稳。

Hook 怎么写：本机 settings.json 真实片段

下面每一行都是从 ~/.claude/settings.json 原样抠出来的

FILE · ~/.claude/settings.json · 用户级 (项目级放 .claude/settings.local.json)

```
REAL · "hooks" { ... } VERBATIM

"PostToolUse": [
  // ① Prettier · TS/JS 改完自动格式化
  { "matcher": "tool == \"Edit\" &&
    tool_input.file_path matches \"\\.(ts|tsx|js|jsx)$\"",
    "hooks": [{ "type": "command",
      "command": "node -e ..prettier..." } ] },
  // ② 9 语言源文件改动 → 触发安全扫描
  { "matcher": "tool == \"Edit\" &&
    file_path matches \"\\.(ts|tsx|js|jsx|py|go|java|cpp|c|rs)$\"",
    "hooks": [{ "command":
      "node ../hooks/auto-security-check.js" } ] },
  // ③ 包管理命令执行后 → build-fix 兜底
  { "matcher": "tool == \"Bash\" &&
    tool_input.command matches \"(npm|yarn|pnpm|uv|pip|cargo|go build)\"",
    "hooks": [{ "command":
      "node ../hooks/auto-build-fix.js" } ] },
  // ④ SQL / DDL 文件改动 → DB review
  { "matcher": "tool == \"Edit\" &&
    file_path matches \"\\.(sql|ddl)$\"",
    "hooks": [{ "command":
      "node ../hooks/auto-db-review.js" } ] }
],
// ⑤ 会话结束 → 扫 debug 语句遗漏
"Stop": [{ "matcher": "*",
  "hooks": [{ "command":
    "node ../hooks/check-console-log.js" } ] } ] }
```

① POSTTOOLUSE · PRETTIER

TS/JS 改完自动格式化

Why：风格偏移 review 时吵最多 · 交 hook 零争议

② POSTTOOLUSE · AUTO-SECURITY

9 种语言源文件触发安全扫

Why：密钥 / SQL 注入别等 commit · 写完就查

③ POSTTOOLUSE · AUTO-BUILD-FIX

包管理命令后 build-fix 兜底

Why：依赖变化 → 类型错 / 破坏性升级 自动挂修

④ POSTTOOLUSE · AUTO-DB-REVIEW

SQL / DDL 文件改动触发 DB review

Why：schema 改动风险最高 · 每次都走一遍

⑤ STOP · CHECK-CONSOLE-LOG

会话结束扫 debug 语句遗漏

Why：AI 有时忘删 · hook 是最后一关

matcher 关键字：tool == "Edit" 锁工具 · tool_input.file_path matches "正则" 锁文件 · tool_input.command matches "模式" 锁 Bash。
命令本身可以是 shell / node / python 脚本，stdin 里是完整 tool_input JSON。

跨栈契约对齐：四套类型，一份真相

Nacos 是配置真相源, AI 是契约对齐官

FIELD	JAVA	GO	PYTHON	TS
tenant_id	@NotNull String	string (json:"")	pydantic.UUID4	string (zod)
created_at	LocalDateTime	time.Time	datetime	string ISO8601
error_code	enum ErrorCode	fmt.Errorf("...")	IntEnum	type union

THE DRIFT PROBLEM

一套语义、四套写法。Java 加字段 → Go orchestrator 透传不知道 → Python 兼容层还在用旧版 → TS 前端类型报错。人眼对四端，稳定漂

AI 的价值：用 review preset (security/types/behavior/edge/docs 5 维) 让 agent 一次扫完四端 → 产出 drift 报告 → ralph-loop 续跑修齐。人只管裁决"保留 / 修 / 显式 deferral"。

质量的度量：纪律在做功

讲质量不能只讲感觉 · 给你看数字

34

MEMORY ENTRIES

每一条都是曾经的故事 · 一次写入，全员受益

772

AGENT CALLS

125 次 team-impl/debug 并行审 · 大幅收敛盲区

30

RALPH-LOOP STARTS

12 次 cancel · 60% 自收敛
不用人管

100%

DEFERRAL 有原文

本项目已记录的 deferral 都有出处可追

BEFORE / AFTER · 纪律 × 质量对照

部署	→ 漏 Nacos 同步 · 500 错误 → 一句话走 4 步 SOP, 不漏
跨栈契约	→ 人眼对四栈漂移 → review preset 五维自动扫, silent drift 大幅减少
长任务	→ 中途忘 · 半途而废 → ralph-loop 续跑到 completion-promise 满足
console.log	→ 混进提交 · 污染日志 → Stop hook 扫全部改动 · 自动拦截
review 处理	→ 全修一轮 3 倍工期 → ROI 分桶 + 显式 deferral, 下次审不重复

核心等式：纪律条数 × hook 命中率 = 你不再需要重复解释的次数 = 你省下来的注意力预算。

质量陷阱 · 反模式五则

别人的经验里最值钱的，永远是错误

× 坑

把所有规则塞进超长 CLAUDE.md，读入上下文就满

✓ 改

拆 memory 小文件 + 索引 · 按需加载

× 坑

主 agent 亲自做 security/db/e2e 全套审查 · 漏还容易爆上下文

✓ 改

sub-agent 并行 · 主只做 调度 + 汇总

× 坑

review finding 全部修完才收工，周期拉 3 倍

✓ 改

ROI 分桶 + 显式 deferral 留痕，下轮不重复扑

× 坑

让 AI 自己改生产 allowlist / origin push / K8s 资源

✓ 改

memory：涉生产必须等确认，AI 只准备不执行

× 坑

ralph-loop 一句"全部修完"就跑 · 跑偏了也不知道

✓ 改

completion-promise 写具体：字段名/接口数/测试通过 — 可字面校验

THE CORE IDEA

AI 不是替代你， 是

纪律松的人，用上 AI 只会更快地产出低质量代码。

纪律清晰的人，用上 AI 能一个人扛住
一整个微服务平台。

放大你的纪律。

— 一人多栈的方法论

OVER TO YOU

Q&A

问啥都行 — 场景 / 坑 / memory 怎么写 / sub-agent 怎么编排

欢迎带来的

你正在做的项目 & 想让 AI 接手的
环节

最想听到的

"在我这儿行不行" 的具体拆解

会后继续

slides + memory 模板 共享

Thanks · **autobots**工作台 · Claude Code 分享 · 2026.04